



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Semestrální práce

Assembler procesoru architektury MISC

Jakub Vokoun

Obsah

1	Zadání	2
2	Analýza úlohy	3
2.1	Dopředná reference	3
3	Popis implementace	5
3.1	core	6
3.1.1	args	6
3.1.2	common	6
3.1.3	memory	8
3.2	io	8
3.3	data_structures	9
3.3.1	dataseg	9
3.3.2	codeseg	9
3.3.3	instruction	9
3.3.4	symbol	10
3.4	lexer	10
3.5	parser	10
3.5.1	Použité struktury	11
3.5.2	Gramatika	12
3.6	assembler	14
4	Uživatelská příručka	16
5	Závěr	17
A	Zadání	19

Zadání

1

Naprogramujte v ANSI C přenositelnou konzolovou aplikaci, která bude zajišťovat funkcionalitu velmi jednoduchého (ale plně funkčního) překladače jazyka symbolických adres (JSA) teoretického/virtuálního počítače – nazvěme ho třeba K's Machine (KM) – s procesorem architektury MISC (= Minimal Instruction Set Computer). Vámi vyvinutý program zpracuje zdrojový soubor zapsaný v JSA (nebo krátce a zjednodušeně assembly) procesoru KM a vygeneruje binární spustitelný soubor ve formátu KMX (K's Machine eXecutable) obsahující odpovídající sekvenci instrukcí strojového kódu níže popsaného minimalistického teoretického/virtuálního počítače.¹

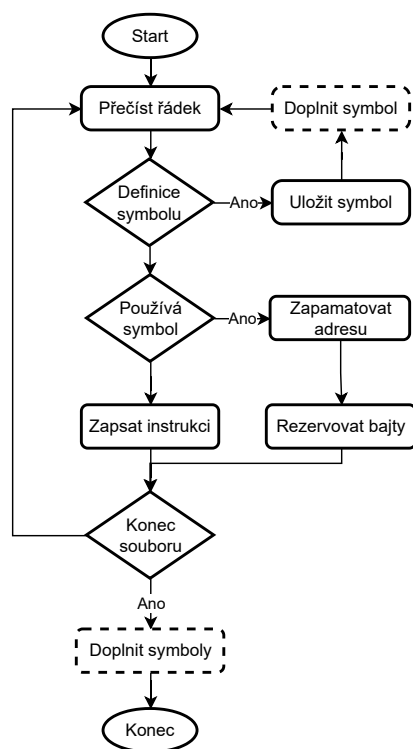
¹Toto je pouze výtah ze zadání. Kompletní znění najdete jako přílohu A na straně 19.

Překlad libovolného assembly souboru do binárního souboru, tedy překlad z jazyka symbolických adres do instrukcí konkrétního procesoru obnáší následující obtíže:

2.1 Dopředná reference

Jazyk symbolických adres je programátory preferovaný proti strojovému (binárnímu) kódu přesně z toho důvodu, který mu dal jméno. Zatímco ve strojovém kódu se mohou vyskytovat pouze adresy (absolutní nebo relativní), v JSA lze definovat a používat symboly, s reprezentující jméno proměnné nebo návěští, které se v době překladu vyhodnotí a do výsledného strojového kódu se zapíše už konkrétní adresy. Tato vlastnost dává programátorům možnost odkazovat se na libovolnou adresu bez toho, aby ji explicitně uváděli, čímž se zabraňuje množství chyb z nepozornosti. Navíc při přidání jediné instrukce ve strojovém kódu je třeba zkontrolovat celý soubor, jelikož všechny adresy následující po této instrukci mají změněnou adresu. Tyto problémy jazyk symbolických adres za cenu potřeby překladu zdrojového kódu elegantně odstiňuje od programátorů.

Řešením tohoto problému může být jednopřůchodový algoritmus, který si udržuje v paměti informace o adresách všech symbolů (proměnných a návěští), zároveň ale i informace o všech místech, kde byl použit neznámý symbol. Algoritmus z kontextu pozná počet bajtů které je třeba rezervovat a zaznamená si neznámý symbol. Jakmile narazí na definici nového symbolu, doplní jeho adresu na všechny místa, kde chybí. Alternativně lze během prvního průchodu pouze zaznamenávat všechna místa obsahující symboly a doplnit je všechny až po projití celého souboru.



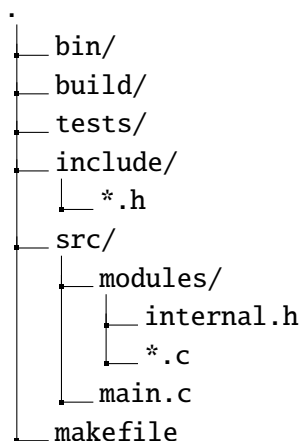
Obrázek 2.1: Zjednodušené zobrazení jednopřechodového překladače. Znázorněny varianty doplnění symbolů v průběhu a na konci pomocí čárkovaně ohraničených boxů.

Jiným řešením je víceprůchodový algoritmus, který v prvních průchodech negeneruje binární kód, pouze zaznamenává pozice symbolů a teprve v dalším (posledním) průchodu přeloží JSA na strojový kód. Tato varianta nevyžaduje tolik paměti RAM, jelikož není potřeba si pamatovat všechny výskyty symbolů. Lépe ilustruje rozdělení analýzy a generace kódu. Nevýhodou je nutnost vícenásobného čtení zdrojového souboru, což může prodloužit dobu kompilace.

Popis implementace

3

Projekt je založen na standardní adresářové struktuře pro *C/C++* projekty¹. V adresáři `include` se nachází veřejné hlavičkové soubory pro jednotlivé části projektu, moduly. Implementace jednotlivých modulů se nachází v adresáři `src`, přičemž jednotlivé zdrojové soubory jsou navíc rozdělené do podadresářů podle funkcionality, které zastávají. Tyto podadresáře v některých případech vyžadují navíc své privátní hlavičkové soubory - takové, které obsahují funkcionalitu společnou pro všechny zdrojové soubory v podadresáři.



Obrázek 3.1: Originální adresářová struktura převzatá z populárních *C/C++* projektů.

¹Validační server pro odevzdávání práce se nedokázal s takto dodaným projektem úspěšně vypořádat. Konkrétně nástroj pro statickou analýzu kódu `splint` vyžadoval hlavičkové soubory ve stejném adresáři se zdrojovými soubory, jinak je nedokázal nalézt. Z toho důvodu byl vytvořen skript pro automatickou transformaci projektu na takový, který odpovídá požadavkům validačního serveru. Pro lepší přehlednost byl ale odevzdán i reverzní skript, který z dodaných souborů zrekonstruuje originální složkovou strukturu. Aplikování skriptu je výrazně doporučeno z důvodů přehlednosti a zajištění toho, že popis projektu odpovídá skutečné adresářové struktuře zdrojových a hlavičkových souborů. Pozor, dodaný `makefile` funguje pouze na strukturu vyžadovanou validačním serverem. Adaptace na standardní adresářovou strukturu je ponechána jako cvičení čtenáři.

Pravidlem v tomto projektu je jasné oddělení funkcí soukromých, vytvořených i používaných v jediném zdrojovém souboru, od funkcí veřejných. Soukromé funkce začínají jednou podtrhávací čarou.

V následujících sekcích rozebereme funkcionalitu, zodpovědnosti a kontrakty jednotlivých modulů. Zároveň i konkrétní důvody rozdělení do podadresářů. Kromě toho na místech vyžadujících to osvětlíme důvody použití maker, práci s pamětí a extenzivního (vhodného) využití volání `goto`.

3.1 core

Moduly v tomto adresáři jsou důležité pro běh celého programu - jsou jeho jádrem, až na modul `memory` nejsou použitelné mimo konkrétní projekt. Moduly z tohoto adresáře jsou často referecovány v celém projektu.

3.1.1 args

Modul je zodpovědný za zpracování argumentů z příkazové řádky. Jejich předání zpět do hlavního toku programu je pomocí datové struktury `Config`, která obsahuje přepínače pro *upovídaný* výstup, výpis jednotlivých zpracovaných instrukcí, zdrojovou a cílovou cestu ². Modul nabízí funkce pro inicializaci a deinicializaci této *datové přepravy*, nikoliv však funkce pro de/alokaci, záměrně se snaží vynutit alokaci pouze na zásobníku. Modul využívá funkcionalitu z modulu pro práci se soubory, aby ověřil existenci uživateli zadaných souborů.

3.1.2 common

Tento modul definuje mnoho maker, které využívají jazykové konstrukce `goto`. Přestože makra mohou být zneužita, záměr je bezpečný. Předpoklad je, že v těle jedné funkce může být více míst, na kterých může ztroskotat. Zároveň každé z těchto míst bude vyžadovat podobné *uklízecí příkazy*, v mnoha případech dokonce stejné.

Zdrojový kód 3.1: Příklad funkce, která by mohla benefitovat z existence maker v modulu `common`.

```
1 int do_smth1(int cond1){  
2     int *buffer = NULL;  
3     int result = 0;  
4  
5     buffer = malloc(10 * sizeof(int));  
6     if (!buffer){  
7         return 0;  
8     }
```

²Použití programu z příkazové řádky je podrobně popsáno v kapitole 4 na straně 16.

```
9
10 if (cond1){
11     buffer[0] = 5;
12 }else{
13     free(buffer);
14     return 0;
15 }
16
17 for (i = 0; i < 10; i++){
18     result += buffer[i];
19 }
20
21 free(buffer);
22 return result;
23 }
```

Nejlépe to jde poznat na funkcích, které alokují paměť, ale jsou atomické a v případě neúspěchu nesmějí způsobovat úniky paměti. Zároveň je vhodné vypsát chybové hlášení. Proto je implementované makro, které přijímá podmínku, návěští a chybovou zprávu. Pokud je podmínka nesplněna, vypíše chybovou zprávu a skočí na dané návěští. To celé je obalené v bezpečnostním `do{}while(0)` obale. Makro má mnoho obalujících maker, které jsou více specifické, například `CLEANUP_IF_FAIL(cond)` předpokládá existenci návěští `cleanup`.

Zdrojový kód 3.2: Hlavní goto makro, které implementuje neúspěchem podmíněný úklid.

```
1 #define GOTO_IF_FAIL_OPTS(cond, label, print_err, ...) \
2     do { \
3         if (!(cond)) { \
4             if ((print_err)) { \
5                 PRINT_ERR(__VA_ARGS__); \
6             } \
7             goto label; \
8         } \
9     } while (0)
```

Modul `common` dále obsahuje globální chybové kódy, definici struktury `Config`, která se hojně používá v celém projektu, funkce a makra pro elegantní chybové výpisy, specifikace KM stroje, KMX binárního souboru, KMA datových typů dle zadání a makry definované konfigurační hodnoty pro jednotlivé části projektu.

Zdrojový kód 3.3: Příklad funkce, která benefituje z existence maker v modulu `common`.

```
1 int do_smth2(int cond1){
2     int *buffer = NULL;
3     int result = 0;
```



```
4
5  buffer = malloc(10 * sizeof(int));
6  CLEANUP_IF_FAIL(buffer);
7
8  CLEANUP_IF_FAIL(cond1);
9  buffer[0] = 5;
10
11  for (i = 0; i < 10; i++){
12      result += buffer[i];
13  }
14
15 cleanup:
16  if (buffer){
17      free(buffer);
18  }
19  return result;
20 }
```

3.1.3 memory

Práce s pamětí v celém projektu probíhá pouze skrze volání funkcí tohoto modulu. Aby se nepletl se standardními funkcemi, je první písmeno všech funkcí nahrazeno písmenem **j**, například místo `free` je `jfree`. Modul obsahuje soukromé počítadlo alokací a na konci funkce `main` je asertován nulový počet alokací.

Při implementaci projektu vyvstala potřeba implementovat funkce, které jsou součástí **POSIX**ového světa, jmenovitě `strdup` a `strndup`. Jelikož tyto funkce pracují s pamětí, byly přidány do tohoto modulu.

Veškerá alokace interně probíhá pomocí funkce `calloc`, která nuluje získanou paměť a tím způsobem chrání uživatele. V projektu se na toto chování ale obecně nespolehá, aby se zabránilo fatálním následům záměny za například z výkonostního pohledu lepší funkci `malloc`.

3.2 io

Veškerá práce s diskem probíhá v režii těchto modulů, které adresář obsahuje dva. Zásadnější z nich je modul `fileutil`, který zajišťuje přenositelnou funkčnost při práci se soubory na strojích typu **linux** i **windows**. Přenositelnost je zajištěna pomocí podmíněného vkládání hlavičkových souborů (`#include`) podle existence preprocesorové proměnné `_WIN32`.

Druhý modul, `output`, využívá těchto a dalších funkcí k tomu, aby zapsal finální binární kód do souboru. Využívá znalosti o struktuře binárního souboru a společně se strukturou vzniklou průchodem JSA kódu, předanou jako argument, je schopen krok po kroku zapsat jednotlivé části algoritmu jako strojový kód.

3.3 data_structures

Čtyři kriticky důležité datové struktury mají každá svůj modul, pro lepší přehlednost a udržitelnost. Ač jsou některé z nich vnitřní implementací velmi podobné jiným, bylo lepší je nechat oddělené pro případ, že by vyvstaly nové požadavky a bylo třeba jednu konkrétní upravit.

3.3.1 dataseg

Modul pro datový segment poskytuje funkce pro vytvoření (alokaci) a uvolnění z paměti. Dále umožňuje vkládat různá data: bajt, slovo, řetězec - podle definice KM stroje. Dokáže zjistit momentální velikost segmentu a nabízí možnost získat konstantní ukazatel na začátek dat, který je potřeba pro zápis do souboru.

Kvůli použití dvouprůchodového algoritmu byly přidány další dvě funkce. Jedna, která pouze simuluje přidávání dat do datového segmentu - pouze zvyšuje počítadlo použitých bajtů (advance). V prvním průchodu je přesně toto použito, aby se odhalilo případné naplnění a přeplnění datového segmentu v KM stroji. Protipól funkce (begin) nastavuje čítač použitých bajtů na nulu a umožňuje tím ten samý datový segment použít jak v prvním, tak i v druhém průchodu.

Datový segment se dynamicky zvětšuje, realokuje, podle násobitele definovaného v modulu `common`. Tato hodnota byla nastavena na dva, aby bylo dosaženo amortizované časové složitosti přidání prvku do segmentu $O(1)$.

3.3.2 codeseg

Kódový segment se v mnohém podobá datovému, podlehlá struktura je stejná, nabízí funkce pro (de)alokaci i funkce `advance` a `begin`. Jinak ale místo vkládání datových typů umožňuje vkládat kód instrukce (op-kód), registr a nebo 32 bitovou hodnotu.

3.3.3 instruction

Modul pro instrukce obsahuje definici struktury `Instruction_Descriptor`, popisovač instrukce. Popisovač instrukce je popsán svým názvem (řetězec), kódem a typy operátorů. Všechny popisovače jsou definované uvnitř zdrojového souboru, tedy neveřejné. Nepřímý přístup k nim nabízí funkce vyhledávající popisovač podle názvu (`find`), která vrací bezpečnou konstantní referenci (bezpečnou proto, že nelze popisovač smazat).

Popisovač instrukce představuje jakýsi vzor bez konkrétních hodnot. V následujícím příkladě budeme uvažovat popisovač jako uspořádanou množinu sestávající z názvu, kódu, typů prvního a druhého operandu. Vezměme konkrétní instrukci `MOV A, 8`. Pro tu existuje popisovač (`"MOV", 10, registr, číslo`). Všimněte si, že nikde

v popisovači není uveden konkrétní registr ani hodnota. Díky tomu lze ten samý popisovač použít i na například instrukci `MOV B, 5`, přestože hodnoty jsou jiné (ale typ instrukce je stejný).

3.3.4 symbol

Tabulka symbolů je nesmírně důležitá pro dvojprůchodový algoritmus. V prvním průchodu se zaznamenávají jména všech symbolů společně s jejich adresami. V druhém průchodu je třeba adresy těchto symbolů získat pouze ze znalosti jejich jmen.

Do jedné tabulky symbolů se v této implementaci vkládají symboly pro proměnné i návěští. Aby se ale při shodě jmen nepřekryly, ukládají se návěští společně s uvozovacím znakem `@`.

Tabulka je implementována jako souvislé pole symbolů, tedy přidání symbolu (probíhající stejným způsobem jako v datovém a kódovém segmentu) je po amortizaci časové složitosti $O(1)$. Neprobíhá ovšem řazení symbolů, lexikografické ani jiné, proto je časová složitost hledání symbolu $O(n)$. Předpokládaná složitost programů používajících tento překladač je dostatečně malá. Pokud by se ale uvažovalo o využití na větších programech, bylo by vhodné zvážit reimplementaci tabulky symbolů jako hashovací tabulky, která nabízí pro obě operace (vklad a výběr) časovou složitost $O(1)$.

3.4 lexer

Účelem modulu `lexer` je, pro jakoukoliv řádku, tedy pro libovoný řetězec znaků ukončený nulovým znakem (`\0`), vytvořit odpovídající pole tokenů. Token je struktura definovaná typem a hodnotou. Hodnota je vždy podřetězec, který odpovídá konkrétnímu tokenu. Uvedu příklad:

Mějme řetězec `swapped DB 2`. Po tokenizaci budou existovat čtyři tokeny: `(IDENTIFIER, "swapped")`, `(DATA_TYPE, "DB")`, `(NUMBER, "2")`, `(EOF, "")`. Vždy typ tokenu jasně rozlišuje, jestli se jedná o proměnnou, datový typ, číslo, konec řádky nebo ještě něco dalšího, zatímco konkrétní hodnota (například jméno proměnné) ještě není známa. Není úkolem lexeru zjistit konkrétní hodnoty. Některé hodnoty jsou vždy stejné (`(LPAREN, "(")`), ale jedná se o menšinu.

3.5 parser

Každý řádek v JSA (respektive jeho reprezentace polem tokenů) patří do jedné z kategorií, které jsou v kódu nazvané `enum Statement_Type`. Úlohou modulu `parser`

je transformovat pole tokenů do struktury nazvané `Parsed_Statement`. Tato struktura obsahuje `Statement_Type` a podle typu se přistupuje ke zbytku struktury.

Tabulka 3.1: Typy řádků.

Statement_Type	význam
NONE	Prázdná řádka, popřípadě komentář.
KMA	Uvozovací direktiva na začátku souboru.
SECTION_DATA	Uvozovací direktiva před deklarací dat.
SECTION_CODE	Uvozovací direktiva před kódovým segmentem.
LABEL_DEF	Definice návěští.
DATA_DECL	Deklarace dat.
INSTRUCTION	Instrukce.
ERROR	Cokoliv se nepovedlo.

3.5.1 Použité struktury

3.5.1.1 Label_Definition

Struktura udávající, že se jedná o definici návěští. Obsahuje řetězec udávající jméno návěští. Přes jednoduchost této struktury byla použita pro lepší semantické pochopení kódu.

3.5.1.2 Instruction_Statement

Popisuje libovolnou instrukci pomocí popisovače instrukce (vizte kapitolu 3.3.3 na straně 9) a konkrétních operandů.

Pomocná struktura `Operand` je popsána typem a specifikací a obsahuje hodnotu závislou na typu. Typ je `enum Operand_Type` z modulu `instruction`. Pokud je operand v popisovači instrukce udán jako `OP_IMM32`, tedy 32 bitová číselná hodnota, může se jednat o pozici návěští v kódovém segmentu, pozici proměnné v datovém segmentu, anebo opravdu o konkrétní číslo. Tato nejasnost je upřesněna specifikací, pomocí hodnoty `enum Operand_Specifier`. Hodnotou v operandu je pak v závislosti na typu a specifikaci jméno registru, číselná hodnota nebo název návěští či proměnné.

3.5.1.3 Data_Declaration

Použitý jazyk symbolických adres umožňuje do jedné proměnné uložit více různých datový typů za sebe, respektive je umožněna deklarace vypadající například

takto: msg DB "Hello, world!", 13, 10, 0, tedy pole bajtů, kde část je deklarována pomocí řetězce a část pomocí numerických hodnot. Tento problém byl vyřešen pomocí oddělení jednotlivých segmentů deklarace. Za segment deklarace je považován výraz oddělený čárkou.

Struktura obsahuje název proměnné, datový typ, pole segmentů a příznak, jsou-li všechny segmenty neinicializované (použit symbol ? v JSA). Každý segment je specifikován typem a v závislosti na něm se přistupuje buď k číselné hodnotě, řetězci, nebo duplikační struktuře (vizte zadání, specifikována hodnotou k opakování a počtem opakování).

3.5.2 Gramatika

Modul parser používá jednoduchou gramatiku, pomocí které rozpoznává typ řádku a kontroluje syntaktické chyby. V tabulce 3.2 je tato gramatika rozepsána. Pracovní verze této gramatiky se nachází i v kódu, jména symbolů byla zde upravena.

Tabulka 3.2: Gramatika použitá ke kategorizaci a syntaktické kontrole řádků. Terminální symboly jsou psané velkými písmeny. Symbol EOF je ukočovací symbol. Novými řádky jsou oddělena různá přepisovací pravidla.

levá strana	pravá strana
line	l-kma l-code l-data l-label l-id l-instr EOF
l-kma	KMA EOF
l-code	CODE EOF
l-data	DATA EOF
l-label	LABEL EOF
l-id	IDENTIFIER id-def

(tabulka pokračuje na další stránce)

Tabulka 3.2 (pokračování z předchozí stránky)

levá strana	pravá strana
id-def	DW, id-dec-dw DB, id-dec-db
id-dec-dw	QUESTION, id-dec-dw-2 NUM, id-dec-dw-2 id-dup-dw
id-dec-dw-2	COMMA, id-dec-dw EOF
id-dup-dw	NUM, DUP, LPAREN, NUM, RPAREN, id-dec-dw-2 NUM, DUP, LPAREN, QUESTION, RPAREN, id-dec-dw-2
id-dec-db	QUESTION, id-dec-db-2 NUM, id-dec-db-2 STRING, id-dec-db-2 id-dup-db
id-dec-db-2	COMMA, id-dec-db EOF
id-db-dup	NUM, DUP, LPAREN, NUM, RPAREN, id-dec-db-2 NUM, DUP, LPAREN, QUESTION, RPAREN, id-dec-db-2
l-instr	INSTRUCTION, instr-rhs
instr-rhs	EOF LABEL, EOF REG, EOF NUM, EOF REG, COMMA, instr-rhs-2
instr-rhs-2	REG, EOF NUM, EOF OFFSET, IDENTIFIER, EOF

Modul parser nad získanou tokenizovanou řádkou volá funkci `grammar_line(pstmt, tokens)`, která z informace uschované v tokenech naplní strukturu `Parsed_Statement` (`pstmt`). Tato funkce patří do *rodiny funkcí*, které jsou v názvu uvozeny prefixem `grammar_`. Všechny tyto funkce fungují na podobném principu: na základě prvního tokenu (v některých případech je to několik tokenů ze začátku) se rozhodnou, jestli se jich předané tokeny týkají. Pokud usoudí že ano, buď zavolají další funkci a předají jí tokeny bez prvního, anebo se jedná už o poslední. Funkce vracejí enum `Err_Grm`, jednu ze tří hodnot: `MATCH` pokud byly tokeny určené

pro tuto funkci, a zároveň všechny volané funkce také vrátily `MATCH`, tedy pokud nikde nenastala žádná chyba. Vrací `NO_MATCH` pokud tokeny neodpovídají tomu, co funkce očekává. A může vrátit `ERROR`, pokud například selhala alokace nebo nastala syntaktická chyba.

Všechny funkce z rodiny *Grammar* odpovídají některému z přepisovacích pravidel. Kontrolují, zda jim předané tokeny odpovídají gramatice. Pokud pravá strana přepisovacího pravidla obsahuje neterminální výraz, příslušná funkce je zavolána na evaluaci předaných tokenů.

Funkce si předávají tokeny a strukturu `Parsed_Statement`, kterou naplňují odpovídajícím způsobem. Vše kromě datové deklarace má předalokované místo přímo v odpovídající struktuře (například ve struktuře pro instrukci je již připravené místo pro dva operandy). Kvůli dynamickému charakteru datové deklarace však program dopředu neví, kolik segmentů je třeba alokovat.

V průběhu datové deklarace se při zpracování každého segmentu pouze zvedá počítadlo datových segmentů. Po zpracování posledního segmentu se na základě počítadla právě jednou alokuje dostatečně velký blok paměti a při procesu vynořování se, se do již připravené paměti zapisují informace o segmentech. To zároveň znamená, že při syntaktické chybě není při vynořování třeba řešit dealokaci paměti, jelikož ještě neexistuje.

3.6 assembler

Pro názornost procesu překladač JSA na binární kód byla zvolena víceprůchodová metoda, konkrétně dvouprůchodová. Struktura `Assembler_Processing` se předává mezi průchody a uchovává získané informace. Úlohou modulu `assembler` je spojit funkcionalitu všech předchozích modulů a přeložit předaný soubor. Trochu konkrétněji, modul musí mít povědomí o celém překládaném souboru (všechny předchozí moduly se zabývaly nanejvýš řádkou).

Funkce sloužící pro průchod souboru jsou téměř totožné, proto byly implementovány s přepínačem pro první či druhý průchod, aby se zabránilo přílišnému opakování kódu (byly ale vytvořeny obalové funkce, pro lepší sémantické pochopení kódu).

První průchod je méně náročný na implementaci a údržbu, jelikož jeho úkolem je pouze projít kód, zkontrolovat případné syntaktické chyby, ale hlavně naplnit tabulku symbolů. Druhý průchod již musí informace z tabulky použít pro naplnění datového i kódového segmentu

Celý modul stále operuje nad jednotlivými řádky, uchovává si ale informaci o již zpracovaných řádcích a na základě této informace kontroluje i strukturu souboru. Například odhalí nepovolenou deklaraci proměnné mimo datový segment.

Do tohoto modulu byla soustředěna většina diagnostických výpisů. Důvodem je vyšší abstrakce a lepší povědomí o pozici v souboru a tudíž i pro uživatele lépe pochopitelné chybové hlášky. Například výpis: `Neexistující token.` nepředává uživateli téměř žádnou informaci, zato výpis obsahující pozici v procházeném souboru, např.: `3,12 lexer error: ...` již pomáhá odhalit chyby.

Uživatelská příručka

4

Ujistěte se, že máte nainstalovaný a funkční nástroj **make**. Dále je zapotřebí překladač **gcc** ve verzi dostatečné pro přeložení **C99** programu. Pokud jste na **Windows**, preferujte použití **MinGW**. Lze použít i jiné překladače, dbejte však na takovou úpravu příslušného **makefile**, aby do proměnné **CC** byl přiřazen Váš překladač a aby v **CFLAGS** nebyly příznaky neshodné s Vaším překladačem. Pro přeložení otevřete terminál (příkazovou řádku) ve umístění kde je příslušný **makefile** a přeložte pomocí příkazu **make**. Po přeložení vznikne spustitelný soubor **kmas.exe**.

Výpis 4.1: Ukázka spuštění programu bez zadání parametrů.

```
1 user@machine: /assembler$ ./kmas.exe
2 Usage: ./kmas.exe <source.kas> [target.kmx] [-v] [-i]
3 user@machine: /assembler$
```

Při spuštění vyžaduje program minimálně jeden argument, cestu ke zdrojovému souboru obsahující JSA kód. Volitelně je možné přidat cestu výstupního binárního souboru a přepínače pro intenzivní výpis diagnostických informací a výpis všech zpracovaných instrukcí.

Po spuštění programu neprobíhá žádná interakce s uživatelem. Program však na standardní výstup vypisuje diagnostické informace, zpracované instrukce - pokud byl spuštěn s příslušným přepínačem. Pokud by došlo k chybě, bude vypsána na **stderr** a program se ukončí s příslušnou návratovou hodnotou. Seznam všech návratových hodnot a jejich významů se nachází v zadání (příloha A na straně 19).

Při implementaci jsem se snažil dbát na dekompozici, nejen funkcí a algoritmů, ale i souborů. Proto je projekt dělen do více adresářů¹, podle mě známých projektových adresářových struktur.

Dekompozice (a doufám, že je to opravdu ten důvod) způsobila, že při jednoduchém porovnání mám i třikrát více řádek (kódu i komentářů) než mí přátelé (jiná zadání, stejný předmět), o počtu souborů nemluvě. Měl jsem snahu vytvořit na sobě nezávislé moduly (kromě funkcí v modulech `memory`, `common` a možná `io`), které by bylo možné použít i v budoucnosti při práci na jiných projektech.

Při práci mi dělalo radost vytvářet makra a nejen pomocí nich se snažit přiblížit vyšším programovacím jazykům. Například pro struktury vytvářet funkce snažící se být konstruktor či destruktor. Ty se pak za použití makra `CLEANUP_IF_FAIL` (viz kapitola 3.1.2 na straně 6) používají na konci funkcí, čímž se snaží napodobit **RAII** techniku, hojně využívanou například v `C++`.

Samostatná výzva byla zprovoznit projekt na validačním serveru. Problém byl s programem **Splint** a pokud správně rozumím chybovým hlášením, tak očekává, že hlavičkové soubory se budou nacházet buď v systémové cestě (`PATH`) anebo v aktuální cestě (vedle zdrojového souboru). Nejjednodušším čistým řešením bylo vytvořit skript, který převede mojí adresářovou strukturu na prostou jednoúrovňovou. Aby se zabránilo jmenným kolizím, upravuje se název souboru podle adresářů, ve kterých se nachází. Tímto se omlouvám za způsobené vrásky při kontrole, přibalil jsem skript, který obnoví mojí (rozumně vypadající) adresářovou strukturu (případně není problém poslat ZIP archiv).

Po zjištění, že přepínač kompilátoru `-Wall` neaktivuje všechna varování jsem se pokusil je dodat manuálně, ale nejsem si jistý, zda jsem některý omylem nevynechal. Přestože to vypadá jako možná až příliš paranoidní přístup, několikrát se stalo, že mě před katastrofou (kterou bych se snažil odhalit dny) zachránil jeden nenápadný přepínač.

Programování v jazyku C má své kouzlo, až překvapivě moc mě to baví. Níz-

¹ Myslím originální projekt, který není upraven pro validační server.

koúrovňová manipulace s pamětí, kde všechno je jenom číslo bez vyšší abstrakce je přitažlivé. Ale doplatil jsem na to, když jsem se loni snažil reimplementovat všechny možné algoritmy a pomocí maker je udělat *genericke* a pustil ze zřetele cíl semestrální práce a pak ji nestihl odevzdat.

Zadání

A

Následující strany obsahují kompletní a nezkrácené zadání semestrální práce, včetně přílohy.

ZADÁNÍ SEMESTRÁLNÍ PRÁCE ASSEMBLER PROCESORU ARCHITEKTURY MISC

Zadání

Naprogramujte v ANSI C přenositelnou¹ **konzolovou aplikaci**, která bude zajišťovat funkcionality velmi jednoduchého (ale plně funkčního) **překladače jazyka symbolických adres (JSA) teoretického/virtuálního počítače** – nazvěme ho třeba K's Machine (KM) – s procesorem architektury MISC (= Minimal Instruction Set Computer). Vámi vyvinutý program zpracuje zdrojový soubor zapsaný v JSA (nebo krátce a zjednodušeně *assembly*) procesoru KM a vygeneruje binární spustitelný soubor ve formátu KMX (K's Machine eXecutable) obsahující odpovídající sekvenci instrukcí strojového kódu níže popsaného minimalistického teoretického/virtuálního počítače².

Assembler se bude spouštět příkazem

```
X:\>kmas.exe3 <program_v_jsa.kas>[<spustitelný_soubor[.kmx]>] [-v] [-i] 
```

Symbol `<program_v_jsa>` představuje povinný parametr – název vstupního zdrojového souboru v jazyce symbolických adres (*assembly*) procesoru KM. Přípona `.kas` (= K's Machine *Assembly*), identifikující soubor se zdrojovým kódem v JSA procesoru KM, musí být vždy uvedena.

Symbol `<spustitelný_soubor>` pak představuje nepovinný parametr, kterým je jméno výstupního binárního spustitelného souboru, do kterého se bude ukládat přeložená sekvence strojových instrukcí procesoru KM. Není-li druhý, nepovinný, parametr uveden, nechť program směřuje výstup do souboru pojmenovaného stejně jako vstupní soubor, ovšem s příponou `.kmx` (místo přípony `.kas` vstupního souboru).

Nepovinný přepínač `'-v'` (= verbose) slouží k aktivaci režimu intenzivního výpisu diagnostických informací. Co přesně bude assembler v tomto režimu vypisovat, je zcela na programátorovi a není to touto specifikací předepsáno.

Nepovinný přepínač `'-i'` (= instructions) slouží k aktivaci režimu výpisu každé jednotlivé přeložené instrukce ze zdrojového textu programu. Formát výpisu není předepsán a slouží zejména k tomu, aby si programátor mohl zkontrolovat, že jím vyvíjený assembler pracuje správně. Doporučený formát výpisu vypadá takto: `'L50: DEC C at CS:150'`. Znamená, že na řádce 50 nalezl assembler instrukci `'DEC'`, jejímž parametrem je registr `'C'` a opkód této instrukce uložil do kódového segmentu výsledného spustitelného programu na adresu 150. Můžete ale zvolit i jiný formát, validátor přesnou podobu výpisu nekontroluje.

Úkolem Vámi vyvinutého programu tedy bude:

1. Načíst prvním parametrem určený vstupní soubor se zdrojovým kódem programu v JSA procesoru KM.
2. Zpracovat všechny deklarační příkazy a podle jejich konkrétní podoby vyhradit odpovídající počet bytů (tedy provést alokaci paměti) v datovém segmentu spustitelného souboru formátu KMX.

¹Je třeba, aby bylo možné Váš program přeložit a spustit na PC s operačním prostředím Win32/64 (tj. operační systémy Microsoft Windows NT/2000/XP/Vista/7/8/10/11) a s běžnými distribucemi Linuxu (např. Ubuntu, Debian, Red Hat, atp.). Server, na který budete Vaši práci odevzdávat a který ji otestuje, má nainstalovaný operační systém Debian GNU/Linux 11 (bullseye) s jádrem verze 5.10.0-32-amd64 a s překladačem gcc 10.2.1-6.

²Návrh tohoto teoretického/virtuálního počítače vychází konceptuálně z Turingova teoretického počítače *Universal Computing Machine* a následně prvních realizovaných počítačů *Manchester Baby*, *Manchester Mark 1*, *EDSAC*, *ENIAC*, apod. Ovlivněn byl částečně samozřejmě i „současností“ v podobě procesoru Intel 8086.

³Přípona `.exe` je povinná i při sestavení v Linuxu, zejm. při automatické kontrole validačním systémem.

3. Uložit adresy všech objektů v datovém segmentu do vnitřních struktur assembleru tak, aby bylo později možné nahradit symbolické vyjádření adres (návěští) objektů/cílových pozic skoků v parametrech strojových instrukcí konkrétními adresami v datovém/kódovém segmentu.
4. Zpracovat všechny instrukce programu, přeložit je na sekvence bytů s odpovídajícími opkódy následované případnými parametry (absolutními hodnotami či signaturami registrů).
5. Uložit sekvence strojových instrukcí do kódového segmentu.
6. Po sestavení obou segmentů (kódového i datového) vypočítat absolutní adresy skoků v kódovém a objektů v datovém segmentu a všechny symbolické adresy (návěští) nahradit konkrétními absolutními adresami.
7. Sestavené segmenty uložit do výstupního souboru ve formátu KMX a doplnit údaje do jeho hlavičky, tak aby vzniknul korektní, emulátorem počítače KM vykonatelný spustitelný soubor.

Váš program může být během testování spuštěn například takto (v operačním systému Windows):

```
X:\>kmas.exe "E:\My Work\Test\mersenne.kas" mersenne.kmx
```

nebo takto:

```
X:\>kmas.exe "E:\My Work\Test\primes.kas" build\primes.kmx
```

nebo pouze takto:

```
X:\>kmas.exe bubble.kas
```

První z uvedených příkladů spuštění by měl vést k vytvoření výstupního spustitelného souboru `mersenne.kmx` v tomtéž adresáři, kde se nachází zdrojový soubor `mersenne.kas`. Druhý příklad vytvoří výstupní spustitelný soubor `primes.kmx` v adresáři `build`, který je podadresářem adresáře, ve kterém se nachází zdrojový soubor `primes.kas`. Při vykonání třetího příkladu bude výstup směřován do souboru `bubble.kmx`, umístěného ve stejném adresáři jako zdrojový soubor.

Spuštění v UNIXových operačních systémech (GNU/Linux, FreeBSD, macOS, atp.) může vypadat např. takto:

```
$/kmas.exe /home/user/work/primes.kas primes.kmx
```

Uvažujte všechny možné specifikace (uvedení úplné cesty, relativní cesty, atp.) jak vstupního, tak nepovinného výstupního souboru, které jsou přípustné v daném operačním systému. Pokud nebude na příkazové řádce uveden alespoň jeden parametr (tj. jméno vstupního souboru se zdrojovým kódem programu v assembleru procesoru KM), vypíše chybové hlášení a stručný návod k použití programu (v angličtině). Návrátová hodnota funkce `int main(...)` bude v tomto případě 1.

Hotovou práci odevzdejte v jediném archivu typu ZIP prostřednictvím automatického odevzdávacího a validačního systému. Postupujte podle instrukcí uvedených na webu předmětu. Archiv necht obsahovat všechny zdrojové soubory potřebné k přeložení programu, **makefile** pro Windows i Linux (pro překlad v Linuxu připravte soubor pojmenovaný **makefile** a pro Windows **makefile.win**) a dokumentaci ve formátu PDF vytvořenou v typografickém systému $\text{\LaTeX}$, resp. $\text{\LaTeX}$. **Bude-li některá z částí chybět, validační systém Vaši práci odmítne.**

Podrobné informace k odevzdání práce najdete na webové stránce předmětu Programování v jazyce C na adrese URL <https://www.kiv.zcu.cz/studies/predmety/pc/index.php#work>.

Popis činnosti programu

Program funguje jako *assembler*⁴ níže popsaného teoretického počítače KM s procesorem architektury MISC. Načítá parametrem předaný vstupní soubor se zdrojovým kódem v JSA počítače KM, provede jeho lexikálně-syntaktickou analýzu a tou získané informace (potřebné pro sestavení obsahu datového a kódového segmentu budoucího spustitelného souboru) převede na sekvenci bytů datového segmentu a sekvenci bytů odpovídající instrukcím strojového kódu a jejich parametrům. Obě binární sekvence následně uloží do výstupního souboru ve formátu KMX, popsaném v sekci Popis formátu KMX na str. 5, který je vykonatelný teoretickým počítačem KM (kdyby existoval) nebo jeho emulátorem.

Assembler při překladu analyzuje jednotlivé příkazy (v částech⁵ definujících obsah datového segmentu začínajících klíčovým slovem `‘.DATA’`) a strojové instrukce (v částech definujících obsah kódového segmentu začínajících klíčovým slovem `‘.CODE’`⁶). Komentáře se mohou vyskytovat kdekoli ve zdrojovém textu, uvozeny jsou znakem `‘;’` (středník) a končí vždy koncem řádky, tj. nemohou být vloženy „dovnitř“ kódu. Assembler komentáře při překladu ignoruje.

Příkazy definující obsah datového segmentu assembler přímo **provádí**, a tím modifikuje velikost a obsah datového segmentu budoucího spustitelného souboru, který je v průběhu překladu udržován v interních strukturách assembleru.

Příkazy pro definici obsahu datového segmentu

V popisu datového segmentu (sekce začíná vždy klíčovým slovem `‘.DATA’`, končí začátkem jiné sekce, a v programu se může vyskytnout vícekrát) lze opakovaně použít pouze následující příkazy definující objekty:

Tabulka 1: Příkazy pro definici objektů v datovém segmentu.

Příkaz	Popis
DWORD	Definuje proměnnou typu <i>double word</i> , tj. 4-bytové/32-bitové celé číslo v kódu s dvojkovým doplňkem, tj. odpovídající typu <code>signed long int</code> jazyka C.
DW	Zkrácený zápis příkazu DWORD.
BYTE	Definuje proměnnou typu <i>byte</i> , tj. 1-bytové/8-bitové celé číslo v kódu s dvojkovým doplňkem, tj. odpovídající typu <code>char</code> jazyka C.
DB	Zkrácený zápis příkazu BYTE.
DUP	Příkaz pro opakované (duplicate) vložení hodnoty do pole prvků daného typu.

Jiné primitivní datové typy než 8-bitový byte deklarovaný příkazem `‘BYTE’` a 32-bitové znaménkové celé číslo deklarované příkazem `‘DWORD’` procesor KM nepodporuje (viz sekce Popis počítače KM na str. 5)!

Příklady a jejich vysvětlení:

- (i) `var1 DWORD ?` — vytváří v datovém segmentu proměnnou s názvem `var1` o šířce 32 bitů, hodnota není určena, tj. proměnná není inicializovaná. Znak `?` za deklarátorem datového typu znamená, že proměnná není inicializovaná a může obsahovat libovolnou hodnotu.
- (ii) `arr1 DW 0, 1, 2, 3, 4` — vytváří v datovém segmentu pole 32-bitových celých čísel s názvem `arr1`, prvky pole jsou čísla 0, 1, 2, 3 a 4. Za deklarátorem datového typu může následovat výčet hodnot (které odpovídají danému datovému typu) oddělených čárkami, které inicializují jednotlivé prvky deklarovaného pole.

⁴https://en.wikipedia.org/wiki/Assembly_language

⁵V jednom zdrojovém kódu jich může být více a také tam nemusí být žádná.

⁶V jednom zdrojovém kódu jich může být více a vždy tam musí být alespoň jedna.

- (iii) `arr2 DWORD 100 DUP(?)` — vytváří v datovém segmentu pole 32-bitových celých čísel o 100 prvcích s názvem `arr2`, prvky pole nejsou inicializované. Příkaz *počet-opakování* `DUP(hodnota)` určuje velikost pole daného bazového typu a hodnotu, kterou budou všechny prvky pole inicializovány. Význam je tedy vlastně *počet-opakování* $\times$ *zDUPlikuj* zadanou hodnotu.
- (iv) `str1 DB "Hello, world!", 0` — vytváří v datovém segmentu pole znaků inicializované jako znakový řetězec ukončený znakem s ASCII hodnotou 0 (tj. tzv. *null-terminated string*, který slouží k ukládání znakových řetězců v jazyce C). Inicializace může být provedena buď čárkami odděleným seznamem hodnot nebo v uvozovkách uzavřenými řetězci znaků (každý z nich reprezentuje jeden byte výsledného pole). Obě techniky inicializace lze libovolně kombinovat (viz ukázka níže).
- (v) `str2 BYTE 20 DUP(0), 1` — vytváří v datovém segmentu pole 20 znaků inicializovaných hodnotou 0, za těmito 20 byty bude bezprostředně následovat jeden byte s hodnotou 1. Inicializátory lze tedy různě kombinovat, každý inicializátor je pak od následujícího oddělen čárkou.

Ukázka definice datového segmentu:

```

1 .KMA
2 .DATA
3     x DWORD ?           ; proměnná 'x' typu 32-bitové číslo, neinicializovaná
4     y DW 10             ; proměnná 'y' typu 32-bitové číslo, inicializovaná na hodnotu 10
5     z DW 100 DUP(65)    ; pole 100 čísel, inicializované na hodnotu 65
6     z2 DW 1, 2, 3, 4, 5 ; pole 5 čísel s hodnotami 1, 2, 3, 4 a 5
7     arr DWORD 20 DUP(?) ; pole 20 čísel, neinicializované
8     msg DB "Hello, world!", 13, 10, 0 ; řetězec následovaný znaky CR+LF a #0
9     grt DB "Hi!", 13, 10, "Hello!", 13, 10, 0 ; řetězec se dvěma řádkami
10 .CODE
11     MOV     A, 1
12     STOR    A, OFFSET x
13     ...

```

Popis datového segmentu končí začátkem jiného segmentu (tedy zřejmě kódového). Jakmile se tedy na následující řádce vyskytne příkaz `.CODE`, definice datového segmentu končí a začíná definice kódového segmentu. Definice obou segmentů (datového i kódového) může být ve zdrojovém textu programu v JSA více, mohou se libovolně střídát, při sestavení výstupního spustitelného programu assemblerem jsou všechny případně oddělené definice jednotlivých segmentů spojeny v jediný datový a jediný kódový segment. Ty jsou následně uloženy do výstupního souboru ve formátu KMX.

Příkazy pro definici obsahu kódového segmentu

V sekcích definujících obsah kódového segmentu⁷ uvozených klíčovým slovem `.CODE` se nacházejí zejména instrukce procesoru KM následované svými případnými parametry. Přehled všech instrukcí teoretického/virtuálního procesoru KM a jejich možných parametrů najdete v sekci Popis teoretického počítače KM na str. 5.

Kromě instrukcí se ve zdrojovém textu kódového segmentu mohou vyskytovat tzv. *návěští*, tedy označení míst, na která je možno skákat prostřednictvím instrukcí podmíněných a nepodmíněných skoků. Návěští mají pevně daný formát: Vždy začínají znakem `'@'` (zavináč), za kterým následuje identifikátor, který splňuje požadavky, kladené na identifikátory v jazyce C, tj. může obsahovat velká i malá písmena, číslice (přičemž číslici nesmí začínat) a znak `'_'` (podtržítko).

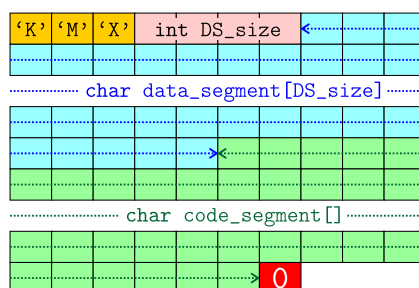
Jediným dalším speciálním příkazem, který se může vyskytnout ve zdrojovém textu kódového segmentu, je `'OFFSET'`. Tento příkaz zajišťuje dodání adresy svého argumentu, kterým musí být objekt z datového segmentu. Např. příkaz JSA ve tvaru `'MOV A, OFFSET var1'` znamená, že se do registru `'A'` vloží adresa proměnné `'var1'` v datovém segmentu, tj. počet bytů od počátku datového segmentu k pozici, kde je uložen první z bytů kódujících tuto proměnnou.

⁷Opět platí, že ve zdrojovém textu programu může být těchto sekcí více, na různých místech.

Ukázku rozsáhlejšího kódu v JSA procesoru KM najdete v příloze Ukázka JSA a strojového kódu počítače KM na str. 11.

Popis formátu KMX spustitelného souboru počítače KM

Soubor ve formátu KMX (= K's Machine Executable) je určen k uložení vykonatelného programu ve strojovém kódu pro teoretický počítač KM. Struktura souboru je velmi jednoduchá a zachycuje ji obr. 1. Soubor je sekvence 8-bitových bytů (datový typ `char` jazyka C). Na začátku souboru je



Obrázek 1: Grafické znázornění uspořádání údajů v binárním souboru formátu KMX.

signatura 3 znaků 'K', 'M' a 'X'. Přítomnost této signatury kontroluje emulátor počítače KM při spuštění programu – pokud v souboru není, hlásí emulátor chybu a ukončí se.

Bezprostředně po signatuře následuje 32-bitové celé číslo (datový typ `int` jazyka C), které představuje délku datového segmentu v bytech. Poté je v souboru KMX uložen obsah datového segmentu programu. Za posledním bytem datového segmentu bezprostředně následuje první byte kódového segmentu. Kódový segment je ukončen bytem s hodnotou 0 – ale **pozor**: byte s hodnotou 0 se může vyskytovat i kdekoliv v datovém nebo kódovém segmentu (může tam být jako argument některé z instrukcí).

Popis teoretického/virtuálního počítače KM s procesorem architektury MISC

Teoretický počítač KM, resp. jeho procesor, je 32-bitový, tzn. šířka registrů (všech bez výjimky) je 32 bitů. Všechna čísla, která lze ukládat do registrů a zpracovávat instrukcemi procesoru, jsou celá, 32-bitová v kódu s dvojkovým doplňkem, tj. odpovídají typu `signed long int` jazyka C. Jiné nativní datové typy tento procesor nezná. Endian procesoru je malý, tzn. nižší řády čísel jsou uloženy na nižších adresách v paměti (stejně jako u procesorů Intel 80x86).

Procesor KM nabízí programátorovi 2 všeobecné střadače (*Accumulators*) 'A' a 'B', a jeden čítač 'C' (= *Counter*). Dále jsou k dispozici dva indexové registry 'S' (= *Source*) a 'D' (= *Destination*), které slouží zejména k indexování dat v datovém segmentu (ale lze je použít i jako střadače).

Virtuální počítač je harvardské architektury, tzn. instrukce vykonávaného kódu jsou v samostatném kódovém segmentu (*Code Segment*, CS), zatímco data jsou v samostatném datovém segmentu (*Data Segment*, DS) paměti. Velikost CS i DS je 256 KB. Počítač má dále pro jednoduchost samostatný 16 KB segment pro zásobník (*Stack Segment*, SS). Pozice právě vykonávané instrukce v kódovém segmentu je určena obsahem registru 'IP' (= *Instruction Pointer*), který ale není programátorovi přístupný (tedy neexistuje žádná instrukce, která by jeho obsah mohla přímo změnit, a tak nemůže být ani parametrem jakékoliv instrukce pro přesun dat). Pozici vrcholu zásobníku jednoznačně určuje obsah registru 'SP' (= *Stack Pointer*). Ten lze měnit jako

kterýkoliv jiný registr pomocí instrukcí MOV, ovšem s tím, že nesprávně provedená změna může mít katastrofální následky pro další vykonávání programu.

Každá instrukce strojového kódu našeho virtuálního počítače zabírá v paměti (v CS) právě jeden byte. Za tímto bytem, který specifikuje instrukci (podle tabulky 3), může (ale nemusí, podle druhu instrukce) následovat jeden či více bytů parametrů dané instrukce. Typicky např. registr, který příslušná instrukce modifikuje, je specifikován dalším bytem v kódovém segmentu podle tabulky 2. Tzn. např. zápis instrukce v assembleru 'MOV B, 16' bude převeden na strojový kód v podobě sekvence bytů (hexadecimálně) '10 02 10 00 00 00'. Byte '10' představuje tzv. *opcode* instrukce MOV (= *Move*, přesun; viz tabulka 3), byte '02' pak parametrické určení dotčeného registru, tedy 'B'. Následují čtyři byty argumentu, tedy 32-bitového čísla s hodnotou 16.

Tabulka 2: Určení registru, je-li parametrem instrukce.

Registr	Kód (hexadecimálně)
A	01
B	02
C	03
D	04
S	05
SP	06

Instrukční sada

Instrukční sada procesoru teoretického počítače KM je úplně popsána tabulkou 3. V popisu každé instrukce může být užito následujících symbolů: (i) *im32* (= *immediate*) znamená 32-bitový argument bezprostředně následující v kódovém segmentu za příslušnou instrukcí; (ii) *reg* (= *register*) znamená, že za instrukcí následuje 1 byte určující, se kterým registrem bude instrukce provedena podle tabulky kódů registrů, tab. 2.

Tabulka 3: Instrukční sada teoretického procesoru KM architektury MISC.

Instrukce (zápis v JSA)	Kód (byty hexa)	Popis činnosti počítače při vykonání instrukce
Pozastavení vykonávání a prázdná instrukce		
HALT	00	Procesor ukončí svoji činnost. Každý korektní program ve strojovém kódu by měl končit touto instrukcí. Pokud ale není poslední instrukcí kódu, nemělo by to vést k havárii.
NOP	90	Procesor vykoná tuto instrukci tím, že neprovede nic (kromě zvýšení IP o 1).
Instrukce pro přesun dat		
MOV <i>reg</i> , <i>im32</i>	10 <i>reg im32</i>	Vloží bezprostředně následující 32-bitové číslo do registru <i>reg</i> .
MOV <i>reg_d</i> , <i>reg_s</i>	11 <i>reg_d reg_s</i>	Vloží obsah registru <i>reg_s</i> do registru <i>reg_d</i> .
MOVSD	12	Načte 32-bitovou hodnotu z datového segmentu na adrese dané obsahem registru S a uloží ji do datového segmentu na adresu danou obsahem registru D, tj. DS[D] ← DS[S].
LOAD <i>reg</i> , <i>im32</i>	13 <i>reg im32</i>	Do registru <i>reg</i> uloží 32-bitové číslo, které se nachází v datovém segmentu na adrese dané operandem instrukce, tedy 32-bitovou hodnotou bezprostředně následující instrukci, tj. <i>reg</i> ← DS[<i>im32</i>].
LOAD <i>reg_d</i> , <i>reg_s</i>	14 <i>reg_d reg_s</i>	Do registru <i>reg_d</i> uloží 32-bitové číslo, které se nachází v datovém segmentu na adrese dané obsahem registru <i>reg_s</i> , tj. <i>reg_d</i> ← DS[<i>reg_s</i>].
STOR <i>reg</i> , <i>im32</i>	15 <i>reg im32</i>	Z registru <i>reg</i> vezme 32-bitové číslo a uloží ho do datového segmentu na adresu danou operandem instrukce, tedy 32-bitovou hodnotou bezprostředně následující instrukci, tj. DS[<i>im32</i>] ← <i>reg</i> .

STOR reg_s, reg_d	16 $reg_s reg_d$	Z registru reg_s vezme 32-bitové číslo a uloží ho do datového segmentu na adresu danou obsahem registru reg_d , tj. $DS[reg_d] \leftarrow reg_s$.
Instrukce pro práci se zásobníkem		
PUSH reg	20 reg	Zvýší hodnotu uloženou v registru SP (= <i>Stack Pointer</i> , ukazatel na pozici vrcholu zásobníku) o 4 a následně na nový vrchol uloží obsah registru reg , tj. $SP \leftarrow SP + 4$; $SS[SP] \leftarrow reg$.
POP reg	21 reg	Přečte z vrcholu zásobníku 32-bitovou hodnotu a vloží ji do registru reg , načež sníží hodnotu registru SP o 4, tj. $reg \leftarrow SS[SP]$; $SP \leftarrow SP - 4$.
Aritmetické instrukce		
ADD $reg, im32$	30 $reg im32$	Přičte k obsahu registru reg 32-bitovou hodnotu bezprostředně následující instrukci, tj. $reg \leftarrow reg + im32$.
ADD reg_d, reg_s	31 $reg_d reg_s$	Přičte k obsahu registru reg_d obsah registru reg_s , tj. $reg_d \leftarrow reg_d + reg_s$.
SUB $reg, im32$	32 $reg im32$	Odečte od obsahu registru reg 32-bitovou hodnotu bezprostředně následující instrukci, tj. $reg \leftarrow reg - im32$.
SUB reg_d, reg_s	33 $reg_d reg_s$	Odečte od obsahu registru reg_d obsah registru reg_s , tj. $reg_d \leftarrow reg_d - reg_s$.
MUL $reg, im32$	34 $reg im32$	Vynásobí obsah registru reg 32-bitovou hodnotou bezprostředně následující instrukci, tj. $reg \leftarrow reg \times im32$.
MUL reg_d, reg_s	35 $reg_d reg_s$	Vynásobí obsah registru reg_d obsahem registru reg_s , tj. $reg_d \leftarrow reg_d \times reg_s$.
DIV $reg, im32$	36 $reg im32$	Vydělí obsah registru reg 32-bitovou hodnotou bezprostředně následující instrukci, tj. $reg \leftarrow reg / im32$.
DIV reg_d, reg_s	37 $reg_d reg_s$	Vydělí obsah registru reg_d obsahem registru reg_s , tj. $reg_d \leftarrow reg_d / reg_s$.
INC reg	38 reg	Zvýší obsah registru reg o 1, tj. $reg \leftarrow reg + 1$. Může dojít k přetečení (považuje se za běžný stav, nikoliv chybu).
DEC reg	39 reg	Sníží obsah registru reg o 1, tj. $reg \leftarrow reg - 1$. Může dojít k podtečení (dtto).
Logické instrukce		
AND $reg, im32$	40 $reg im32$	Provede logický součin (konjunkci) obsah registru reg s 32-bitovou hodnotou bezprostředně následující instrukci, tj. $reg \leftarrow reg \wedge im32$.
AND reg_d, reg_s	41 $reg_d reg_s$	Provede logický součin (konjunkci) obsah registru reg_d s obsahem registru reg_s , tj. $reg_d \leftarrow reg_d \wedge reg_s$.
OR $reg, im32$	42 $reg im32$	Provede logický součet (disjunkci) obsah registru reg s 32-bitovou hodnotou bezprostředně následující instrukci, tj. $reg \leftarrow reg \vee im32$.
OR reg_d, reg_s	43 $reg_d reg_s$	Provede logický součet (disjunkci) obsah registru reg_d s obsahem registru reg_s , tj. $reg_d \leftarrow reg_d \vee reg_s$.
XOR $reg, im32$	44 $reg im32$	Provede exkluzivní logický součet (nonekvivalenci) obsah registru reg s 32-bitovou hodnotou bezprostředně následující instrukci, tj. $reg \leftarrow reg \oplus im32$.
XOR reg_d, reg_s	45 $reg_d reg_s$	Provede exkluzivní logický součet (nonekvivalenci) obsah registru reg_d s obsahem registru reg_s , tj. $reg_d \leftarrow reg_d \oplus reg_s$.
NOT reg	46 reg	Nahradí obsah registru reg jeho jedničkovým doplňkem (čili inverzí všech bitů), tj. $reg \leftarrow \neg reg$.
Instrukce pro bitové manipulace		
SHL $reg, im32$	50 $reg im32$	Posune obsah registru reg doleva o počet bitů daný 32-bitovou hodnotou bezprostředně následující instrukci (ovšem z této 32-bitové hodnoty se pro posun bere v potaz pouze 5 nejméně významných bitů), tj. $reg \leftarrow reg \ll im32$.

SHL reg_d, reg_s	51 $reg_d reg_s$	Posune obsah registru reg_d doleva o počet bitů daný hodnotou registru reg_s (ovšem z této 32-bitové hodnoty se pro posun bere v potaz pouze 5 nejméně významných bitů), tj. $reg_d \leftarrow reg_d \ll reg_s$.
SHR $reg, im32$	52 $reg im32$	Posune obsah registru reg doprava o počet bitů daný 32-bitovou hodnotou bezprostředně následující instrukci (ovšem z této 32-bitové hodnoty se pro posun bere v potaz pouze 5 nejméně významných bitů), tj. $reg \leftarrow reg \gg im32$.
SHR reg_d, reg_s	53 $reg_d reg_s$	Posune obsah registru reg_d doprava o počet bitů daný hodnotou registru reg_s (ovšem z této 32-bitové hodnoty se pro posun bere v potaz pouze 5 nejméně významných bitů), tj. $reg_d \leftarrow reg_d \gg reg_s$.
Porovnávací instrukce		
CMP $reg, im32$	60 $reg im32$	Porovná obsah registru reg s 32-bitovou hodnotou bezprostředně následující instrukci a nastaví vnitřní příznaky procesoru tak, aby šlo podle výsledku porovnání provést skok, tj. if ($reg = im32$) ...
CMP reg_1, reg_2	61 $reg_1 reg_2$	Porovná obsah registru reg_1 s obsahem registru reg_2 a nastaví vnitřní příznaky procesoru tak, aby šlo podle výsledku porovnání provést skok, tj. if ($reg_1 = reg_2$) ...
Instrukce skoků		
JMP $im32$	70 $im32$	Provede skok v kódovém segmentu na adresu danou 32-bitovou hodnotou bezprostředně následující instrukci, tj. $IP \leftarrow im32$.
JMP reg	71 reg	Provede skok v kódovém segmentu na adresu danou obsahem registru reg , tj. $IP \leftarrow reg$.
JE $im32$	72 $im32$	Provede relativní skok v kódovém segmentu směrem k vyšším (při kladné hodnotě $im32$) či nižším (při záporné hodnotě $im32$) adresám o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, je-li výsledkem předchozí instrukce CMP rovnost ($= \text{Jump if Equal}$), tj. if (...) $IP \leftarrow IP + im32$.
JNE $im32$	73 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, je-li výsledkem předchozí instrukce CMP nerovnost ($= \text{Jump if Not Equal}$), tj. if (...) $IP \leftarrow IP + im32$.
JG $im32$	74 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, byl-li při provedení předchozí instrukce CMP první operand větší než druhý ($= \text{Jump if Greater}$), tj. if (...) $IP \leftarrow IP + im32$.
JGE $im32$	75 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, byl-li při provedení předchozí instrukce CMP první operand větší nebo roven druhému ($= \text{Jump if Greater or Equal}$), tj. if (...) $IP \leftarrow IP + im32$.
JNG $im32$	76 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, nebyl-li při provedení předchozí instrukce CMP první operand větší než druhý ($= \text{Jump if Not Greater}$), tj. if (...) $IP \leftarrow IP + im32$.
JL $im32$	77 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, byl-li při provedení předchozí instrukce CMP první operand menší než druhý ($= \text{Jump if Less}$), tj. if (...) $IP \leftarrow IP + im32$.
JLE $im32$	78 $im32$	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, byl-li při provedení předchozí instrukce CMP první operand menší nebo roven druhému ($= \text{Jump if Less or Equal}$), tj. if (...) $IP \leftarrow IP + im32$.

JNL <i>im32</i>	79 <i>im32</i>	Provede relativní skok v kódovém segmentu o 32-bitovou hodnotou bezprostředně následující instrukci tehdy, nebyl-li při provedení předchozí instrukce CMP první operand menší než druhý (= <i>Jump if Not Less</i>), tj. $\text{if } (\dots) \text{ IP} \leftarrow \text{IP} + \text{im32}$.
Instrukce na podporu podprogramů		
CALL <i>im32</i>	80 <i>im32</i>	Předá řízení podprogramu, jehož vstupní bod (počáteční adresa v CS, tj. adresa první instrukce tohoto podprogramu) je dán 32-bitovou hodnotou bezprostředně následující instrukci. Oproti instrukci JMP musí CALL před skokem na danou adresu uložit do zásobníku tzv. návratovou adresu, čili adresu instrukce, která bezprostředně následuje po této instrukci CALL, tj. $\text{PUSH } (\text{IP} + 5)$; $\text{IP} \leftarrow \text{im32}$.
CALL <i>reg</i>	81 <i>reg</i>	Předá řízení podprogramu, jehož vstupní bod je dán hodnotou v registru <i>reg</i> . Oproti instrukci JMP musí CALL před skokem na danou adresu uložit do zásobníku tzv. návratovou adresu, čili adresu instrukce, která bezprostředně následuje po této instrukci CALL, tj. $\text{PUSH } (\text{IP} + 2)$; $\text{IP} \leftarrow \text{reg}$.
RET	82	Vrátí řízení z podprogramu zavolaného instrukcí CALL volajícím (pod)programu. Adresu pro návrat získá instrukce RET ze zásobníku, kam ji uložila instrukce CALL, tj. $\text{POP } \text{val}$; $\text{IP} \leftarrow \text{val}$.
Emulátorové instrukce pro vstup a výstup údajů		
OUTD <i>reg</i>	F0 <i>reg</i>	Vypíše na konzoli emulátoru jako celé dekadické číslo hodnotu uloženou v registru <i>reg</i> .
OUTC <i>reg</i>	F1 <i>reg</i>	Vypíše na konzoli emulátoru jeden znak, jehož ASCII hodnota je umístěna v registru <i>reg</i> (pro výpis znaku se bere v úvahu pouze nejméně významných 8 bitů).
OUTS <i>reg</i>	F2 <i>reg</i>	Vypíše na konzoli emulátoru řetězec znaků (sekvence ASCII znaků ukončený znakem s ASCII hodnotou 0), jehož adresa (počátku) v DS je umístěna v registru <i>reg</i> .
INPD <i>reg</i>	F3 <i>reg</i>	Načte z konzole emulátoru celé dekadické číslo a to uloží do registru <i>reg</i> .
INPC <i>reg</i>	F4 <i>reg</i>	Načte z konzole emulátoru jeden znak a ten uloží (jeho ASCII hodnotu) do registru <i>reg</i> .
INPS <i>reg</i>	F5 <i>reg</i>	Načte z konzole emulátoru řetězec znaků (sekvence ASCII znaků ukončený znakem s ASCII hodnotou 0) a ten uloží do DS na adresu určenou obsahem registru <i>reg</i> .

Pokud si u některé instrukce nejste z popisu jisti, jak se přesně má chovat, prozkoumejte chování takové (nebo podobné instrukce) u některého existujícího procesoru, např. Intel 8086 (kterým je instrukční sada procesoru KM silně inspirovaná).

Specifikace výstupu programu

Program je konzolová aplikace, tj. ovládá se pouze parametry předanými při spuštění v příkazové řádce (konzoli/terminálu). V průběhu překladu zdrojového kódu v JSA do strojového kódu emulovaného teoretického počítače KM se žádná interakce s uživatelem nepředpokládá, kromě případných diagnostických výpisů, pokud je uživatel požaduje (použije přepínač ‘-v’ (= verbose) či ‘-i’ (= instructions) na příkazové řádce).

Pokud nastane závažný chybový stav, který nedovoluje assembleru pokračovat v činnosti, označuje to uživateli srozumitelným chybovým hlášením v anglickém jazyce vypsáním na konzoli (do standardního proudu `stderr`) a následně vynuceným ukončením programu s předáním návratové hodnoty podle tabulky 4. Jiným než výše popsaným způsobem assembler během své činnosti s uživatelem neinteraguje.

Tabulka 4: Návrátové kódy programu v případě chyby.

Popis chybového stavu	Návrátová hodnota funkce int main()
ERR_NO_ERROR: Bez chyby/korektní ukončení činnosti assembleru.	0
ERR_INVALID_INPUT_FILE: Neplatný název vstupního souboru, nebo soubor neexistuje či nelze otevřít pro čtení.	1
ERR_INVALID_OUTPUT_FILE: Neplatný název výstupního souboru, nebo soubor nelze vytvořit či otevřít pro zápis.	2
ERR_SYNTAX_ERROR: Syntaktická chyba ve zdrojovém textu programu v JSA. Assembler narazil např. na instrukci, která není uvedena v tab. 3 nebo u instrukce chybí parametr nebo je naopak uveden parametr u instrukce, která žádný nevyžaduje.	3
ERR_FILE_ACCESS_FAILURE: Soubor (vstupní či výstupní) je nepřístupný, nelze s ním pracovat (protože je např. otevřen ve výhradním režimu jiným procesem).	4
ERR_OUT_OF_MEMORY: Assembleru došla operační paměť. To by se stát nemělo, ale pokud ano, bude program reagovat tímto návratovým kódem.	5
ERR_UNRESOLVED_REFERENCE: Assembler narazil na situaci, kdy je v kódu použit symbol (např. identifikátor návěští, funkce, objektu v datovém segmentu, apod.), který nelze nalézt a převést na adresu.	6
ERR_CODE_SEGMENT_TOO_LARGE: Kódový segment je příliš velký a nevejde se do limitu 256 KB daného specifikací počítače KM.	7
ERR_DATA_SEGMENT_TOO_LARGE: Datový segment je příliš velký a nevejde se do limitu 256 KB daného specifikací počítače KM.	8
Ostatním chybovým stavům můžete přiřadit návratové kódy dle svého uvážení.	

Užitečné informace

Níže uvedené zdroje informací je možné (ale nikoliv nezbytně nutné) využít při řešení úlohy. Protože konstrukce assembleru je věc v informatice celkem běžná, lze k tomu nalézt velké množství různých dokumentace např. za pomoci vyhledávače Google:

1. Informace o JSA a assembleru na Wikipedii – <https://en.wikipedia.org/wiki/Assembly>
2. Instrukční sada CPU Intel 8086 – https://en.wikipedia.org/wiki/X86_instruction_listings
3. Syntaktická analýza rekurzivním sestupem – https://en.wikipedia.org/wiki/Recursive_descent_parser
4. Brian Callahan: Starting an assembler – <https://briancallahan.net/blog/20210408.html>
5. Michal Brandejs: *Mikroprocesory Intel 8086 – 80486*. Grada, 1991. ISBN 80-85424-27-4.

Řešení úlohy je zcela ve vaší kompetenci – zvolte takové algoritmy a techniky, které podle vás nejlépe povedou k cíli. Ačkoli vypadá zadání na první pohled velmi složité, lze řešení implementovat programem o přibližně 2000 řádcích zdrojového kódu (přednášející to zvládl na 1847 řádek).

Příloha: Ukázka JSA a strojového kódu počítače KM

V níže uvedené ukázce je naprogramován v assembleru počítače KM algoritmus řazení *Bubble Sort*.
V komentáři za každou instrukcí je její podoba ve strojovém kódu (po bytech, hexadecimálně).

```

1 .KMA
2 .DATA
3     swapped DWORD ?           ; items swapped in the inner loop, DS:[0]
4     array DWORD 20 DUP(?)     ; the data to be sorted, DS:[4]
5 .CODE
6     MOV     A, 1               ; 10 01 01 00 00 00
7     STOR    A, OFFSET swapped ; 15 01 00 00 00 00
8 @LBubbleWhileBeg:             ; <--- CS:[12]
9     LOAD    A, OFFSET swapped ; 13 01 00 00 00 00
10    CMP     A, 1               ; 60 01 01 00 00 00
11    JNE     @LBubbleWhileEnd   ; 73 62 00 00 00
12 @LBubbleForInit:             ; <--- CS:[29]
13    MOV     C, 19              ; 10 03 13 00 00 00
14    MOV     A, 0               ; 10 01 00 00 00 00
15    STOR    A, OFFSET swapped ; 15 01 00 00 00 00
16    MOV     S, OFFSET array    ; 10 05 04 00 00 00
17    MOV     D, OFFSET array    ; 10 04 04 00 00 00
18    ADD     D, 4               ; 30 04 04 00 00 00
19 @LBubbleForBeg:               ; <--- CS:[65]
20    LOAD    A, S               ; 14 01 05
21    LOAD    B, D               ; 14 02 04
22    CMP     A, B               ; 61 01 02
23    JNG     @L01               ; 76 12 00 00 00
24    STOR    B, S               ; 16 02 05
25    STOR    A, D               ; 16 01 04
26    MOV     A, 1               ; 10 01 01 00 00 00
27    STOR    A, OFFSET swapped ; 15 01 00 00 00 00
28 @L01:                         ; <--- CS:[97]
29    DEC     C                  ; 39 03
30    ADD     S, 4               ; 30 05 04 00 00 00
31    ADD     D, 4               ; 30 04 04 00 00 00
32    CMP     C, 0               ; 60 03 00 00 00 00
33    JG      @LBubbleForBeg     ; 74 C7 FF FF FF
34 @LBubbleForEnd:               ; <--- CS:[122]
35    JMP     @LBubbleWhileBeg   ; 70 0C 00 00 00
36 @LBubbleWhileEnd:             ; <--- CS:[127]
37
38 @LOutputForInit:              ; <--- CS:[127]
39    MOV     C, 20              ; 10 03 14 00 00 00
40    MOV     S, OFFSET array    ; 10 05 04 00 00 00
41 @LOutputForBeg:               ; <--- CS:[139]
42    LOAD    A, S               ; 14 01 05
43    OUTD    A                  ; F0 01
44    ADD     S, 4               ; 30 05 04 00 00 00
45    DEC     C                  ; 39 03
46    CMP     C, 0               ; 60 03 00 00 00 00
47    JG      @LOutputForBeg     ; 74 E8 FF FF FF
48 @LOutputForEnd:               ; <--- CS:[157]
49    HALT                      ; 00

```

1101001
101011000011100010 1100001
101011010101 10

11010011101101001
0110000110101
111000101011101